DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

VALORES NULOS – NULL SAFETY

- Em kotlin qualquer tipo de variável pode receber um valor nulo, mas para isso devemos informar na declaração da variável que ela permite nulo colocando uma ? após o tipo da variável:

```
fun main() {
    var b: Int

    b = null
}
```

```
fun main() {
    var b: Int?

    b = null
}
```

VALORES NULOS – NULL SAFETY

- Mesmo que seja informado um valor para uma variável que aceita nulo, o kotlin não deixa fazer contas com essa variável porque seu valor pode ser nulo.
- Para resolver isso podemos utilizar !! para falar para o kotlin não fazer a verificação e deixar essa responsabilidade para o programador.

```
fun main() {
    var b: Int? = 5

    var a = b * 2
}
```

```
fun main() {
    var b: Int? = 5

    var a = b!! * 2
}
```

EXCEÇÕES

- Quando tem algum erro no código o Kotlin (como qualquer outra linguagem) não deixa o programa ser compilado até que o erro seja corrigido.
- Mas muitas vezes o código não tem erros, mas ao executar o programa um erro é gerado, como uma divisão por zero ou tentar transformar uma string com letras ou vazia em uma variável numérica.
- Esses erros apresentados durante a execução do programa são chamados de exceções.

EXCEÇÕES

- Quando uma exceção ocorre a execução do programa é abortada, a linguagem exibe uma mensagem de erro com o nome da exceção que ocorreu, o nome da função onde ela ocorreu e em qual linha do código esta o problema.
- O programador pode prever essas exceções e tratar a mesma para que o programa não seja abortado, permitindo que o usuário corrija o problema e continue executando o programa.
- Para tratar as exceções utilizamos o comando try catch

EXCEÇÕES –TRY CATCH

```
try {
        [código que será executado]
}
catch ( [exceção] ) {
        [código a ser executado caso a exceção acima ocorra]
}
finally {     /*o finally é opcional*/
        [código que será executado sempre, independe de ter uma
         exceção ou não]
}
```

EXCEÇÕES –TRY CATCH

- Note que no código ao lado ele grifa de amarelo avisando que tem um erro de divisão por zero, mas deixa o programa ser executado

```
fun main() {
    val a = 5
    val b = 0

    println( a / b )
}
```

- A exceção apresentada é ArithmeticException

```
C:\Users\alex\.jdks\openjdk-20.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains
Exception in thread "main" java.lang.ArithmeticException Create breakpoint : / by zero
    at Aula1Kt.main(aula1.kt:5)
    at Aula1Kt.main(aula1.kt)

Process finished with exit code 1
```

EXCEÇÕES –TRY CATCH

- Podemos tratar a exceção da seguinte forma:

```
fun main() {
    val a = 5
    val b = 0

    try {
        println( a / b )
    } catch ( e: ArithmeticException ) {
        println( "Impossivel dividir por zero!")
    }
}
```

- Agora ele não apresenta mais erros, e executa o código até o final:

```
C:\Users\alex\.jdks\openjdk-20.0.2\bin\java.exe "-javaagent:C:\Program Fi
Impossivel dividir por zero!

Process finished with exit code 0
```

EXCEÇÕES –TRY CATCH

- Outro exemplo:

```
fun main() {
    var num1: String
    var num2: String
    var continua: String
    var resultado: Int

    do {
        print( "Digite o primeiro numero: ")
        num1 = readln()
        print( "Digite o segundo numero: ")
        num2 = readln()

        try {
            resultado = num1.toInt() / num2.toInt()
            println( "a divisão de $num1 por $num2 é $resultado")
        } catch ( e: ArithmeticException ) {
            println( "Impossivel dividir por zero!")
        }

        print( "fazer outra conta (s/n)? ")
        continua = readln()

    } while ( continua == "s" )
}
```

EXCEÇÕES –TRY CATCH

- O programa não da erro se for digitado 0 no segundo valor, ele exibe a mensagem "Impossível dividir por zero!" e continua a execução.
- Mas se for deixado um dos números em branco, ele exibe a seguinte mensagem de erro:

```
Digite o primeiro numero:
Digite o segundo numero: 5
Exception in thread "main" java.lang.NumberFormatException Create breakpoint : For input string: ""
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:675)
    at java.base/java.lang.Integer.parseInt(Integer.java:781)
    at Aula1Kt.main(aula1.kt:14)
    at Aula1Kt.main(aula1.kt)

Process finished with exit code 1
```

EXCEÇÕES –TRY CATCH

- Podemos tratar essa exceção da seguinte forma:

```
try {
    resultado = num1.toInt() / num2.toInt()
    println( "a divisão de $num1 por $num2 é $resultado")
} catch ( e: ArithmeticException ) {
    println( "Impossivel dividir por zero!")
} catch ( e: NumberFormatException ) {
    println( "Favor informar um número válido!")
}
```

- Um comando try pode ter quantos catch forem necessários.
- Sempre que uma exceção acontece, ele mostra qual o nome dela para que possamos tratar a mesma dentro do try.